

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/326171835>

Securing Input Data of Deep Learning Inference Systems via Partitioned Enclave Execution

Preprint · July 2018

CITATIONS

0

READS

230

7 authors, including:



Zhongshu Gu

IBM Research

21 PUBLICATIONS 234 CITATIONS

[SEE PROFILE](#)



Heqing Huang

IBM T.J Watson Research Center

19 PUBLICATIONS 345 CITATIONS

[SEE PROFILE](#)



Ankita Lamba

IBM

8 PUBLICATIONS 0 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Hermes: A Mobile App [View project](#)



Software Defined Networks [View project](#)

Securing Input Data of Deep Learning Inference Systems via Partitioned Enclave Execution

Zhongshu Gu[†], Heqing Huang[†], Jialong Zhang[†], Dong Su[†],
Ankita Lamba[†], Dimitrios Pendarakis[‡], Ian Molloy[†]
[†]IBM Research, [‡]IBM Cognitive Systems

Abstract

Deep learning systems have been widely deployed as backend engines of artificial intelligence (AI) services for their approaching-human performance in cognitive tasks. However, end users always have some concerns about the confidentiality of their provisioned input data, even for those reputable AI service providers. Accidental disclosures of sensitive user data might unexpectedly happen due to security breaches, exploited vulnerabilities, neglect, or insiders. In this paper, we systematically investigate the potential information exposure in deep learning based AI inference systems. Based on our observation, we develop *DeepEnclave*, a privacy-enhancing system to mitigate sensitive information disclosure in deep learning inference pipelines. The key innovation is to partition deep learning models and leverage secure enclave techniques on cloud infrastructures to cryptographically protect the confidentiality and integrity of user inputs. We formulate the information exposure problem as a *reconstruction privacy attack* and quantify the adversary’s capabilities with different attack strategies. Our comprehensive security analysis and performance measurement can act as a guideline for end users to determine their principle of partitioning deep neural networks, thus to achieve maximum privacy guarantee with acceptable performance overhead.

1 Introduction

The recent breakthroughs of deep learning (DL) are catalyzed by unprecedented amounts of data, innovations in learning methodologies, and the emergence of learning-accelerated hardware. DL-based approaches can achieve or surpass human-level performance on computer vision [30, 37, 59], speech recognition [25, 28, 31], machine translation [60], game playing [57, 58], etc. Today major cloud providers offer artificial intelligence (AI) services, e.g., Amazon AI [1], Google Cloud AI [4], IBM Wat-

son [6], and Microsoft Azure [8], powered with DL backend engines to help end users augment their applications and edge devices with AI capabilities. End users of AI services can be individuals or collective entities that represent independent software vendors, enterprises, health facilities, educational institutions, governments, etc.

AI cloud providers generally offer two independent DL services, i.e., training and inference. End users can build customized DL models from scratch by feeding training services with their own training data. In case if end users do not possess enough training data, they can also leverage transfer learning techniques [49] to repurpose and retrain existing models targeting similar tasks. After obtaining their trained models, end users can upload the models, which are in the form of hyperparameters and weights of deep neural networks (DNNs), to inference services — which might be hosted by different AI service providers as of training services — to bootstrap their AI cloud APIs. These APIs can be further integrated into mobile or desktop applications. At runtime, end users can invoke the remote APIs with their input data and receive prediction results from inference services.

Although end users always expect that service providers should be trustworthy and dependable, they may still have some concerns about the data privacy of their inputs. Accidental disclosures of confidential data might unexpectedly occur due to malicious attacks, misoperations by negligent system administrators, or data thefts conducted by insiders. Adversaries with escalated privileges may be able to extract sensitive data from disks (data-at-rest) or from main memory (runtime data) [51–53]. We have observed numerous data breach events [5, 9] in recent years. Similar incidents can also happen to user input data for AI cloud services. In addition, deep learning is often differentiated by processing raw input data, such as images, audio, and video, as opposed to hand-crafted features. This poses more privacy concerns if the input data are leaked or compromised.

To address the data privacy problem on AI cloud, researchers proposed cryptographic-primitives-based approaches [22, 42, 46] to enable privacy-preserving predictions. Although they have made significant performance improvement increasingly, it is still far from being practical to apply such approaches to meet end users’ requirements. The other line of approaches to protecting data privacy are based on distributed machine learning [40, 48, 55], which intended to delegate part of the deep learning functionality to the client sides and transfer masked feature representations to the cloud. However, these approaches complicated program logic and consumed more network bandwidth for client devices, which are not supposed to handle computing-intensive workload. We also observed that Ohrimenko et al. [47] proposed data-oblivious multi-party machine learning algorithms (including neural networks) and leverage Intel Software Guard Extensions (SGX) to make them privacy-preserving. However, the performance (inadequate acceleration for matrix computation and floating-point arithmetic) and memory capacity (the protected physical memory size for Intel Skylake CPU is 128MB¹) constraints of SGX restrict the adaptability of their approach to deeper neural networks trained on large-scale datasets, e.g., ImageNet [17]. Recently, we also aware of ongoing research efforts, such as Chiron [34] and Slalom [62], which intend to address the similar data privacy protection problem in deep learning pipelines by leveraging SGX. Different from prior enclave-based efforts, our approach tends to exploit the intrinsic properties of deep learning model structures as the partitioning principle to achieve optimal security-performance balance.

In this paper, we present *DeepEnclave*, a privacy-enhancing deep learning inference system to mitigate information exposure of sensitive input data in the inference pipelines. The key innovation of *DeepEnclave* is to partition each deep learning model into a *FrontNet* and a *BackNet* by exploiting the layered structure of neural networks. End users are allowed to submit encrypted inputs and encrypted *FrontNet* to our system. We leverage the Intel SGX on cloud infrastructures to enforce enclaved execution of the *FrontNet* and cryptographically protect the confidentiality and integrity of user inputs. Meanwhile, the inference computation of the *BackNet* runs out of secure enclaves and can still benefit from the performance improvement if cloud machines are equipped with DL-accelerated chips.

The key challenge of this approach is to determine the optimal model-specific partitioning points that bal-

ance the privacy protection and performance requirements. We formulate the information exposure problem as a *reconstruction privacy attack* and quantify the adversary’s capabilities with different attack strategies and prior knowledge. We develop a *neural network assessment framework* to quantify the information leakage for the outputs of *FrontNets* with different numbers of layers and automate the process of finding optimal partitioning points for different neural network architectures. We conduct our security measurement on three ImageNet-level deep neural networks, i.e., Darknet Reference Model (17 layers) [2], Extraction Model (28 layers) [3], and DenseNet Model (306 layers) [33], with different network depths and architectural complexity. In addition, by protecting the confidentiality of both user inputs and *FrontNet* models, we ensure that our system can effectively defend against state-of-the-art input reconstruction techniques adaptable to deep neural networks. Our comprehensive security and performance analysis can be used as a guideline for end users to determine their own principle for partitioning DNNs, thus to achieve maximized privacy guarantee with acceptable performance overhead.

2 Motivation

By comparing the inputs and outputs of a deep learning based image classification system, we find that user input data might be unnecessarily exposed in deep learning inference pipelines. We give a motivating example in Figure 1 to empirically demonstrate such information exposure. We input a picture to a 1000-class image classification system with a Convolutional Neural Network (ConvNet) model trained on the ImageNet dataset. The output of the system is the Top-5 prediction class scores of the picture. Just like the idiom saying that “a picture is worth a thousand words”, we can learn rich information from this input picture. It is obvious to tell that this picture was taken in *Rio de Janeiro* (1.a) by identifying the *Sugarloaf Mountain* (1.b). Anyone who is familiar with the geographical layout of *Rio de Janeiro* can also pinpoint *Botafogo* (1.c), *Copacabana* (1.d), and *Guanabara Bay* (1.e). In addition, we can further infer that the picture must be taken at the *Cristo Redentor* (1.g) because this is the only place you can have this specific view of the *Sugarloaf Mountain*. Based on the position (west side) of the sunlight, we can deduce that the time should be around sunset (2.a). Furthermore, if we check the EXIF metadata of this picture, we can verify our previous hypotheses with the GPS coordinates (1.f) and the *DayTimeOriginal* (2.b). We can also obtain the device type (3.a) and its configuration (3.b, 3.c) by reading the metadata. If combined with other contextual information, it is possible to reveal the identity of the person who

¹With memory paging support for Linux SGX kernel driver, the size of enclave memory can be expanded with memory swapping. But swapping on the encrypted memory will significantly affect the performance.

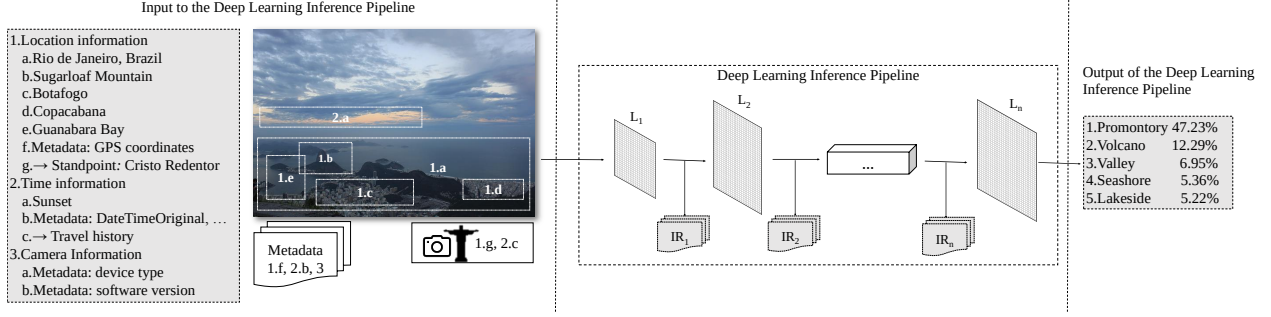


Figure 1: The Information Exposure in a Deep Learning Inference Pipeline

took this picture and recover his or her travel history.

On the contrary, the output of the image classification system only reveals limited information indicating that this picture can be classified as a *promontory*, *volcano*, *valley*, *seashore*, or *lakeshore*, with different confidence values. We consider that there is a *privacy gap* the represents the information discrepancy between the inputs and outputs at the two ends of the deep learning inference pipeline. The gap may disclose users' sensitive information to AI cloud providers, and further the information may possibly be leaked to malicious adversaries.

The computation of a deep neural network is to distill feature representations layer by layer. The process can be formulated as a composited transformation function that maps raw inputs to outputs with a specified target. Each hidden layer performs its own transformation as a sub-function and generates an intermediate representation (IR) as an output. Conceptually, transformation on each hidden layer contributes more or less to make the intermediate representations converge towards the final outputs. Our work is inspired by the research efforts for understanding the internals of deep neural networks [56, 67, 68]. As indicated by Zeiler and Fergus [68], for an image classification ConvNet, the shallow layers respond more to low-level photographic information, such as edges, corners, contours, of the original inputs, while deep layers can represent more abstract and class-specific information related to the final outputs. From the privacy perspective, low-level photographic information can reveal more precise and specific information of the original inputs. Whereas, high-level abstract representations generated in deep layers contain less private information. Therefore, to protect the sensitive information within inputs, we can choose to enclose first several layers of a DNN into an isolated execution environment, which is kept confidential and tamper-resistant to external computing stacks.

3 Problem Definition

Based on the layered structure of deep neural networks, we partition each network into two independent subnet models, i.e., a *FrontNet* and a *BackNet*. Mathematically, a deep neural network can be defined as a function F^* that maps the input $\mathbf{x}$ to the output $\mathbf{y}$, i.e., $\mathbf{y} = F^*(\mathbf{x}; \theta)$. θ stands for the parameters that are learned in the training phase. The function F^* is composed of n (assuming the network has n layers) sub-functions D_i where $i \in [1, n]$. F_i maps the input $\mathbf{x}_i$ to the output $\mathbf{y}_i$ on Layer i . These sub-functions are connected in a chain. Thus $\mathbf{y} = F^*(\mathbf{x}; \theta) = F_n F_{n-1} \dots F_1(\mathbf{x})$. After partitioning the neural network at the m -th layer where $m \in [1, n]$, the function for *FrontNet* can be represented as $\Phi: \mathbb{X} \mapsto \mathbb{IR}$. $\mathbb{X}$ is the input space applicable for a specific deep neural network and $\mathbb{IR}$ is the output space for the intermediate representations. $\mathbf{IR} = \Phi(\mathbf{x}; \theta_\Phi) = F_m F_{m-1} \dots F_1(\mathbf{x})$ and its output IR is the intermediate representation computed out of the *FrontNet*. The function λ for *BackNet* is $\lambda(\mathbf{IR}; \theta_\lambda) = F_n F_{n-1} \dots F_{m+1}(\mathbf{IR})$, in which IR is the input.

We assume that the adversaries might have some background knowledge B for reconstructing the sensitive original input $\mathbf{x}$. The background knowledge includes: (1) the domain knowledge of user inputs, e.g. input file types, natural image priors [43]; (2) the knowledge on the distribution of all bits of $\mathbf{x}$, which can be described by a probability matrix $P = \{\dots, p_{ij}, \dots\}$, where p_{ij} is the probability that the i -th bit of $\mathbf{x}$ takes the value j , $1 \leq i \leq |\mathbf{x}|$ and $j \in \Omega$, Ω is the encoding alphabet, and $\forall i, \sum_j p_{ij} = 1$.

Adversaries aim to reconstruct the inference input $\mathbf{x}$: given an $\mathbf{IR} \in \mathbb{IR}$ of $\mathbf{x}$ and the background knowledge B , adversaries can devise an attack strategy A to return $\tilde{\mathbf{x}}$, the reconstructed version of $\mathbf{x}$. The attack strategy A can span from visually perceiving the intermediate representations to leveraging advanced input reconstruction techniques by approximating the inverse model. The *FrontNet* representation function $\Phi(\cdot)$ is considered to violate the ϵ -privacy for $\mathbf{x}$, if there exists an attack A , back-

ground knowledge B and intermediate representation IR ,

$$\frac{\text{dist}[\mathbf{x}, \tilde{\mathbf{x}} | \tilde{\mathbf{x}} \leftarrow A(B, IR)]}{\text{dist}[\mathbf{x}, \tilde{\mathbf{x}} | \tilde{\mathbf{x}} \leftarrow A(B)]} \leq \varepsilon, \quad (1)$$

where ε is the privacy parameter to bound the distances between $\mathbf{x}$ and $\tilde{\mathbf{x}}$ before and after observing IR and $\varepsilon \in [0, 1]$. The dist measures the distance between an original input $\mathbf{x}$ and a reconstructed input $\tilde{\mathbf{x}}$. Specifically, $\text{dist}[\mathbf{x}, \tilde{\mathbf{x}} | \tilde{\mathbf{x}} \leftarrow A(B)]$ considers that $\tilde{\mathbf{x}}$ is reconstructed only based on adversaries' background knowledge B . Whereas in $\text{dist}[\mathbf{x}, \tilde{\mathbf{x}} | \tilde{\mathbf{x}} \leftarrow A(B, IR)]$, $\tilde{\mathbf{x}}$ is reconstructed based on both the adversaries' background knowledge B and the observed IR . Eq. 1 says that the privacy of the true inference input $\mathbf{x}$ is breached if adversaries can significantly reduce the distance between $\tilde{\mathbf{x}}$ and $\mathbf{x}$ after obtaining the intermediate representation IR of $\mathbf{x}$.

4 Threat Model

In our threat model, we assume that adversaries are able to obtain data from machines of deep learning cloud systems. There are multiple ways for them to achieve that. For example, attackers may exploit some zero-day vulnerabilities to penetrate and compromise the system software of the server. Insiders, such as cloud administrators, can also retrieve and leak data from the servers on purpose. The data can be files on disks or snapshots of physical memory. We assume that adversaries understand the format of the files stored on disks and they are able to locate and extract structured data (of their interest) from memory snapshots. We also expect that adversaries master the state-of-the-art techniques [18, 29, 43] for reconstructing inputs from IR s.

However, we assume that the adversaries cannot break into the perimeters of CPU packages to track the code execution and data flow at the processor level. We do not intend to address the side-channel attacks against Intel SGX in this paper. But in Section 9 we introduce some recent representative SGX side-channel attacks, give an in-depth analysis why the core computation of deep neural networks is still resilient to side channel attacks, and the potential vulnerabilities.

We assume that adversaries do not have access to the training dataset, thus they cannot train a surrogate model. This is a reasonable assumption because end users only need to submit pre-trained models to AI cloud providers, rather than releasing their training dataset. Protecting training data privacy is out of the scope of this paper. If end users depend on third-party training providers to train their models, they may consider privacy-preserving training mechanisms [45–47, 55] to protect training data from being leaked to training providers. Furthermore, we

do not expect that adversaries have access to the whole (or the superset of) inference input datasets. Otherwise, we consider in this scenario the inference data have already been leaked and adversaries only need to determine which samples have been used as inputs, rather than reconstructing the contents of the inputs. Having the privilege to access inference input datasets is not realistic in the general settings of AI cloud services.

In addition, we intend to guarantee the data privacy of *user input data*, but protecting the confidentiality and integrity of *final outputs* of deep learning services is out of the scope of this paper. Here we propose some preliminary solutions for the readers interested in this problem. To protect the prediction results, end users can upload models that only output class ids, rather than meaningful class labels. Therefore, users can interpret the outputs on their local machines without leaking the classification results. In addition, end users can also deploy the DNN output layer, i.e., the last layer along with the softmax activation function, into a secure enclave and deliver the outputs directly to end users via a secure communication channel. To protect the integrity of outputs, end users may leverage statistical sampling methods to validate inference results via a local DNN that shares the same topology as its cloud counterpart.

5 System Design

In order to protect the confidentiality and integrity of user-provisioned inputs, we design *DeepEnclave*, a privacy-enhancing cloud-based deep learning inference system. Here we explain the key components of our system in detail.

5.1 Partitioning of DNNs

As defined in Section 3, the representation function for a *FrontNet* is $IR = \Phi(\mathbf{x}; \theta_\Phi)$ and a *BackNet* is $\mathbf{y}^* = \lambda(\Phi(\mathbf{x}; \theta_\Phi); \theta_\lambda)$. The parameter θ of the original neural network is divided into θ_Φ and θ_λ according to the network partition. The output shape of a *FrontNet* should be compatible with the input shape of its corresponding *BackNet*. We deliver IR as an input to a subsequent *BackNet* and continue the computation to get a result $\mathbf{y}^*$. Given the same input $\mathbf{x}$, we expect that $\mathbf{y}^*$ should be equivalent to $\mathbf{y}$, which is the output of the original neural network before the partition.

5.2 Secure Remote DNN Computation

On the cloud side, the *FrontNet* and inputs from end users should be loaded into a Trusted Execution Environment (TEE) that can guarantee the confidentiality, integrity, and freshness of the protected memory for secure

remote computation. We choose to use the Intel SGX enclave [44] as the TEE in our research prototype, but our approach in principle can also be generalized to other TEEs [12, 36]. With the protection of the memory access control mechanism and memory encryption engine (MEE) of the SGX, all non-enclave accesses from privileged system software or other untrusted components of systems will be denied. Thus the computational process of the user inputs with the *FrontNet* is kept within the perimeter of a specific CPU package and is invisible to the external world. The computation within an enclave is still naturally dedicated to distilling features for specific inference tasks, just exhibiting the same behaviors as its counterpart running out of the enclave. Furthermore, the enclave can attest to remote parties (i.e., the end users of AI cloud services) that the *FrontNet* is running in a secure environment hosted by a trusted hardware platform.

5.3 Confidentiality of Inference Inputs

In order to protect the contents of user inputs from being exposed on cloud servers, end users need to encrypt inputs with their symmetric keys and upload the encrypted files to cloud services. After finishing the remote attestation with the enclave, end users can provision the symmetric keys to the enclave via a secure communication channel. The code inside the enclave then decrypts the user inputs and passes the inputs to the *FrontNet* model, which should have been loaded in the same enclave. In addition, we leverage the Galois Counter Mode (GCM) to achieve authenticated encryption. Thus we can authenticate legitimate end users and render service abusing attacks ineffective. For the adversaries who tend to treat the in-enclave *FrontNet* as a black-box service and query to extract model information, they need to encrypt their inputs with the proper symmetric keys from the legitimate end users. Assuming that end users' keys are not leaked, we can deny serving these illegitimate requests that fail the integrity check and prevent the leakage of *FrontNet* model information, which is considered to be crucial for reconstructing user inputs.

In our initial design, we do not require end users to provide encrypted *FrontNet* models. This can protect the user data privacy if adversaries only intend to infer original inputs from *FrontNet*'s IR outputs. We demonstrate in Section 6.1 that the quality of IR images (we map the IRs back to the pixel space) decay rapidly after passing first few layers. However, advanced adversaries may master the techniques to reconstruct inputs from IRs in neural networks. Although the convolution and pooling operations of ConvNet are not reversible, more powerful adversaries might have access to both the IRs and the model parameters. With some prior knowledge, adversaries can still approximately generate inputs that are

Algorithm 1 Privacy-Enhancing DNN Classification

Input:	<i>img_enc</i> ▷ Encrypted image input <i>fn_cfg</i> ▷ <i>FrontNet</i> neural network configuration <i>fnw_enc</i> ▷ Encrypted <i>FrontNet</i> neural network weights <i>bn_cfg</i> ▷ <i>BackNet</i> neural network configuration <i>bnw</i> ▷ <i>BackNet</i> neural network weights <i>clt</i> ▷ Client key provisioning ip
--------	---

```

1: ##### Within SGX Enclave #####
2: ▷ Encrypted Input/Encrypted FRONTNET
3: function ENCLAVE_LOAD_ENC_MODEL(fn_cfg, fnw_enc, clt)
4:   tls ← ENCLAVE_ATTESTATION(clt)
5:   fnw_key, img_key ← ENCLAVE_GET_KEYS(clt, tls, fnw_l, img_l)
6:   fnw ← ENCLAVE_DECRYPT(fnw_enc, fnw_key)
7:   fn ← ENCLAVE_LOAD_WEIGHTS(fn_cfg, fnw)
8:
9: function ENCLAVE_INFERENCE_ENC_IMG(img_enc)
10:  img ← ENCLAVE_DECRYPT(img_enc, img_key)
11:  ir ← ENCLAVE_NETWORK_INFERENCE(fn, img)
12:  return ir
13:
14: ##### Out of SGX Enclave #####
15: ▷ Encrypted Input/Encrypted FRONTNET
16: function INF_ENC_MODEL_IMG(fn_cfg, fnw_enc, bn_cfg, bnw, img_enc, clt)
17:  eid ← INIT_ENCLAVE()
18:  ENCLAVE_LOAD_ENC_MODEL(eid, fn_cfg, fnw_enc, client)
19:  ir ← ENCLAVE_INFERENCE_ENC_IMG(eid, img_enc)
20:  bn ← LOAD_WEIGHTS(bn_cfg, bnw)
21:  result ← NETWORK_INFERENCE(bn, ir)
22:  return result

```

similar to the original inputs [18, 29, 43]. In order to defend against such *input reconstruction attacks*, we enhance our design to support user-provisioned encrypted *FrontNet*. By protecting the confidentiality of both user inputs and the *FrontNet* model, all state-of-the-art (as far as we known) input reconstruction methods will no longer be effective. We give a detailed analysis on why we can neutralize input reconstruction techniques in Section 6.2.

5.4 Workflow

We summarize the workflow of *DeepEnclave* (using an image classification service as an example) by explaining the steps in Figure 2 and corresponding pseudo-code in Algorithm 1. In this case, an end user can provide both encrypted inputs and a pre-trained model (with an encrypted *FrontNet*).

① The end user needs to partition a model into a *FrontNet* and a *BackNet* (we implement a tool to automate offline model partitioning in our research prototype). The *FrontNet* should be kept in secret and encrypted with a symmetric key from the end user. We do not expect to protect the *BackNet* in our scenario and the configuration and weights of *BackNet* are shared to the cloud provider. As the *BackNet* is not supposed to run inside an SGX enclave for performance constraints, we omit the discussion of the protection of parametric data of the *BackNet* here. But it would be better to use standard encryption mechanisms and protocols to protect the *BackNet* in communication and at rest. In addition to the *FrontNet* model, the end user also needs to encrypt the inputs with

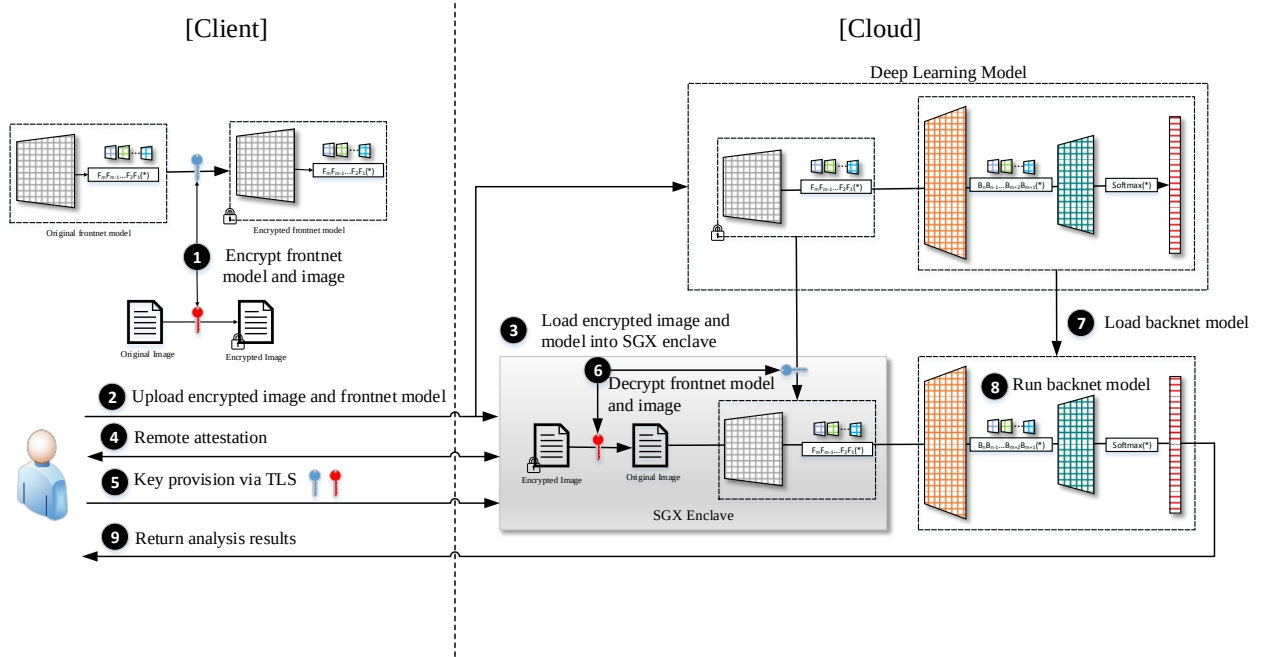


Figure 2: The Workflow of an Image Classification Service via *DeepEnclave*

her symmetric key.

② The end user uploads the encrypted model to the image classification service on the cloud. She only needs to provide the model once to initiate the service. After the service starts, the end user can continuously upload encrypted inputs for classification.

③ On the cloud side, after receiving the encrypted image and the encrypted *FrontNet* model, *DeepEnclave* instantiates an Intel SGX enclave (INIT_ENCLAVE at line 17) and loads the encrypted *FrontNet* model (ENCLAVE_LOAD_ENC_MODEL at line 18) for deep neural network computation into the enclave. Then the cloud service invokes the image classification API function (ENCLAVE_INFERENCE_ENC_IMG at line 19) and securely copy the encrypted image into the enclave as the function argument.

④ The end user and the SGX enclave need to perform remote attestation [10] procedure. The enclave can prove to the end user that it is running on top of a trusted hardware platform with legitimate code/data from a trusted cloud service provider. The detailed description of the standard attestation protocol can be found in an example [7] provided by Intel. Due to the unclear licensing procedure for registering SGX enclave code and the prerequisites for using the *Intel Attestation Server* (IAS), we currently skip this step and instantiate a TLS session directly between the end user and the enclave.

⑤ After creating a secure Transport Layer Security (TLS) communication channel, the end user can provision symmetric keys (ENCLAVE_GET_KEYS at line 5) di-

rectly into the enclave on the cloud.

⑥ Inside the enclave, we verify the integrity of both the model and the input by checking their GCM authentication tags, and decrypt the *FrontNet* model (ENCLAVE_DECRYPT at line 6) and the input (ENCLAVE_DECRYPT at line 10) with the provisioned symmetric keys from the end user. Then we can build a deep neural network based on the *FrontNet* (ENCLAVE_LOAD_WEIGHTS at line 7), pass the decrypted input into the model (ENCLAVE_NETWORK_INFERENCE at line 11), and generate the IR from the *FrontNet*.

⑦ The generated IR is securely copied out of the enclave through a controlled channel of SGX. We build another deep neural network based on the *BackNet* model (LOAD_WEIGHTS at line 20).

⑧ We then pass the IR into the *BackNet* and get the final classification result (NETWORK_INFERENCE at line 21). The final result is an N -dimensional real-value vector that represents a probability distribution over N different possible classes.

⑨ Based on the specification, the deep learning cloud service can choose to return the Top- k classes with their corresponding probabilities back to the end user.

6 Security Analysis

After building the *DeepEnclave* system for partitioned enclave execution, we tend to address the problem of *determining the optimal partitioning points* for deep neural

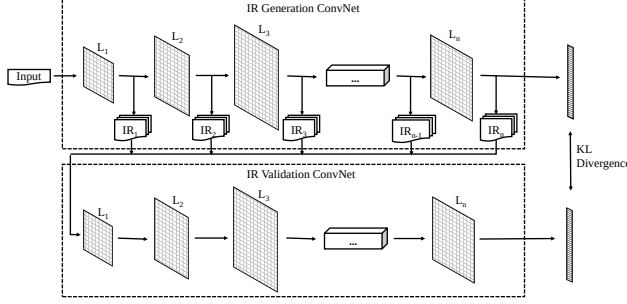


Figure 3: The Architecture of the Neural Network Assessment Framework

networks via a comprehensive security analysis.

Here we simulate two hypothetical adversaries, $\mathcal{A}_1$ and $\mathcal{A}_2$, within the privacy reconstruction attack framework (defined in Section 3) and they tend to uncover the contents of original raw input $\mathbf{x}$ after obtaining IRs out of the enclave. We consider both adversaries have no prior knowledge of input $\mathbf{x}$, i.e., probability matrix P holds the uniform distribution and $\forall i, p_{ij} = \frac{1}{|\Omega|}$, but they have different (from weak to strong) attack strategies A:

$\mathcal{A}_1$: This adversary is able to view IRs generated out of the *FrontNet*. The strategy A is to pick the IR that reveals the most information of the original input. We tend to measure the information exposure by assessing IRs at different partitioning layers of a DNN.

$\mathcal{A}_2$: In addition to viewing the IRs, this more advanced adversary can further master these input reconstruction techniques for deep neural networks. Thus the strategy A of the adversary is to derive an inverse function ϕ^{-1} from Φ and compute $\tilde{\mathbf{x}} = \phi^{-1}(\text{IR})$. The reconstructed $\tilde{\mathbf{x}}$ may leak the information of the original input $\mathbf{x}$. We tend to demonstrate that *DeepEnclave* by design can render such attack ineffective.

6.1 Perception of IRs ($\mathcal{A}_1$)

Based on our threat model, we assume that the adversary $\mathcal{A}_1$ is able to retrieve the IR data of the hidden layers located out of SGX enclaves, even though the IRs may only reside in the computer memory. Therefore, it is crucial to investigate whether this adversary can perceive and infer the contents of the original inputs by viewing the IRs. In ConvNet, IRs are organized in the forms of stacked feature maps. Thus we project all feature maps back to the pixel space and save them as IR images. For example, if a convolutional layer of a model has 64 filters and the output is a $112 \times 112 \times 64$ tensor, we can generate 64 IR images (112 in width and 112 in height) from its output. We conduct experiments for three ImageNet-level deep neural networks, i.e., Darknet Reference Model (17 layers with 5,040 IR images) [2], Extraction Model (28

layers with 12,880 IR images) [3], and DenseNet Model (306 layers with 107,888 IR images) [33].

One method to simulate this adversary is to let human subjects view all IR images and pick the ones that reveal the original input $\mathbf{x}$'s information. However, this task is tedious and error-prone for human beings considering the quantity of IR images they need to inspect and is also difficult to quantify the distance between $\mathbf{x}$ and IRs. Instead, we replace human subjects with another ConvNet (by exploiting ConvNet's approaching-human visual recognition capability) to automatically assess all IR images and identify the ones revealing most input information at each layer. This approach is based on the insight that if an IR image retains similar content as the input image, it will be classified into similar categories with the same ConvNet. By measuring the similarity of classification results, we can deduce whether a specific IR image is visually similar to its original input. End users can further leverage the assessment results to determine the optimal partitioning points for different neural network architectures.

Neural Network Assessment Framework In Figure 3, we present the Dual-ConvNet architecture of our neural network assessment framework. We submit an input $\mathbf{x}$ to the IR Generation ConvNet (IRGenNet) and generate IR_i $i \in [1, n]$. Each IR_i contains multiple feature maps after passing Layer i (L_i). Then we project feature maps to IR images and submit them to the IR Validation ConvNet (IRValNet), which shares the same network architecture/weights as the IRGenNet. The outputs of both ConvNets are N -dimensional (N is the number of classes) probability vectors with class scores. We use the Kullback-Leibler (KL) divergence to measure the similarity of classification results. At each Layer i , we select the IR image with the minimum KL divergence D_{KL} with the input $\mathbf{x}$ to quantitatively measure the $\text{dist}[\mathbf{x}, \text{IR}_i]$: $\forall j \in [1, \text{filter_num}(L_i)]$,

$$\begin{aligned} \text{dist}[\mathbf{x}, \text{IR}_i] &= \min_j (D_{KL}(F^*(\mathbf{x}, \theta) \| F^*(\text{IR}_{ij}, \theta))) \\ &= \min_j \left(\sum_k F^*(\mathbf{x}, \theta)_k \log \frac{F^*(\mathbf{x}, \theta)_k}{F^*(\text{IR}_{ij}, \theta)_k} \right), \end{aligned} \quad (2)$$

where $F^*(\cdot, \theta)$ is the representation function shared by both IRGenNet and IRValNet. To determine the optimal partitioning point for each neural network, we compute $D_{KL}(F^*(\mathbf{x}, \theta) \| \mu)$ where $\mu \sim \text{U}(0, N)$, the uniform distribution of the probability vector and N is the number of classes. This represents that $\mathcal{A}_1$ has no prior knowledge of $\mathbf{x}$ before obtaining IRs and considers that $\mathbf{x}$ will be classified to all classes with equal chance. Based on Eq. 1, we can compute $\delta_i = \frac{\text{dist}[\mathbf{x}, \text{IR}_i]}{D_{KL}(F^*(\mathbf{x}, \theta) \| \mu)}$ and compare δ_i with the user-specified ϵ bound. For example, if the user chooses $\epsilon = 1$, to avoid violating ϵ -privacy, it is safe

to partition at Layer i only if $\delta_i > \varepsilon = 1$. It is worth noting that comparison with the uniform distribution with $\varepsilon = 1$ is a very tight privacy bound for the information exposure. In the real-world scenario, end users can relax the constraint to specify their specific $\varepsilon \in [0, 1]$ bound to satisfy their privacy requirements.

6.1.1 Model Analysis

Darknet Reference Model This is a relatively small neural network for ImageNet classification. Its parameter amount is only 1/10th of AlexNet [37], but it still retains the same prediction performance (Top-1: 61.1% and Top-5: 83.0%). We display the result of our assessment for the first six layers in Figure 4. For each hidden layer, we choose the IR image that has the minimum KL divergence. For example, Layer 1 is a convolutional layer and the most similar IR image to the original input is generated by the 6th filter of this layer. We can still visually infer that the original content in the first several layers, but getting more and more difficult for subsequent layers. In Figure 7, we present the range of KL divergence scores (black columns) for the IR images of all layers except the last three layers, i.e., average pooling, softmax, and cost layers, which do not generate IR images. For example, at Layer 1 the minimum KL divergence is 1.2 and the maximum is 5.9. We also highlight the line for the KL divergence of the uniform distribution, which is 3.88, with regard to $\mathbf{x}$. We can find that after Layer 3, the minimum KL divergence scores approach and surpass the line of uniform distribution’s KL, which indicates that viewing IRs after Layer 3 cannot help $\mathcal{A}_1$ reveal information from the original input anymore. Thus end users can choose to partition the network at Layer 3 and enclose them to run within the enclave.

Extraction Model Compared to the Darknet Reference Model, the Extraction Model is deeper and can achieve higher prediction accuracy (Top-1: 72.5% and Top-5: 90.0%). We present the most similar-to-input IR images of its first six layers in Figure 5 and the KL divergence scores in Figure 8. We can observe a similar phenomenon that after Layer 6, the KL divergence score ranges exceed the KL divergence of uniform distribution. Thus the safe partitioning point for this neural network is at Layer 6.

DenseNet Model In classical ConvNet architectures, each layer only obtains the input from its precedent layer. However, with the increase of network depth, it may lead to the vanishing gradient problem [11, 23]. To address this issue, researchers introduced short paths cross layers to make it practical to train very deep neural networks. The authors of the DenseNet Model [33] introduced the

neural network topology with DenseBlocks. Within each DenseBlock, each layer obtains inputs from all preceding layers and also transfers its own IRs to all subsequent layers. Between two adjacent DenseBlocks, it contains transitional layers to adjust the IR’s size. We find that such special network structures, i.e., DenseBlocks and densely connected layers, can be consistently quantified with KL divergence. We show the KL divergence scores in Figure 9. The DenseNet Model has four DenseBlocks. In each DenseBlock, the minimum KL divergence scores plummet regularly every two layers. The reason is that there exist route layers (after every two convolutional layers) that receive inputs from all preceding layers in the same DenseBlock. For example, the minimum KL divergence of Layer 4 (convolutional layer) is 5.07, while Layer 5 (route layer) drops to 2.49. If we partition in the middle of DenseBlock 1, the following IRs in DenseBlock 1 can still reveal the input’s information. However, there is no densely connected path that *crosses* different DenseBlocks. Although there still exist fluctuations of KL divergence scores in the DenseBlock 2, the scores are significantly larger than layers in DenseBlock 1. In Figure 6, in addition to the Layer 1 and 5, we also display the most similar-to-input IR images at all transitional layers (Layer 21, 59, and 205) between different DenseBlocks. Based on the uniform distribution KL divergence (5.57), the optimal partition point is at Layer 21 (the last layer of DenseBlock 1), or for more cautious end users, they can choose to partition at Layer 59 (the last layer of the DenseBlock 2).

Remarks End users can choose to include fewer layers inside an enclave, which may leak some degree of information, or more layers for stronger privacy guarantee, but may face some performance and usability constraints. As we have shown in the experiments for the three representative deep learning models, different neural network architectures may have different optimal partitioning points. With our neural network assessment framework, end users can test and determine the optimal partitioning layer for their specific deep learning models on local machines before uploading the inputs and models to the deep learning inference services.

6.2 Input Reconstruction Techniques ($\mathcal{A}_2$)

With a stronger adversarial model, we expect that the adversary $\mathcal{A}_2$ may master advanced input reconstruction techniques, hence $\mathcal{A}_2$ aims to first reconstruct the inputs from the intermediate representations. We describe the general input reconstruction problem in deep neural networks formally as follows: the representation function at Layer i of a given DNN is $\Phi_i : \mathbb{R}^{w \times h \times c} \mapsto \mathbb{R}^d$, which transforms the input $\mathbf{x}$ to $\Phi_i(\mathbf{x})$. Given an intermediate

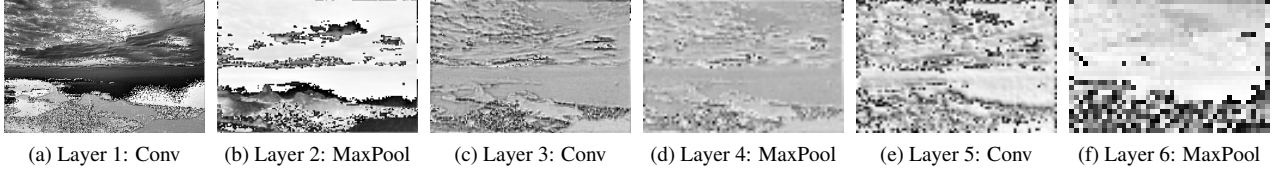


Figure 4: The List of Most Similar-to-Input IR Images at Each Hidden Layer — Darknet Reference Model

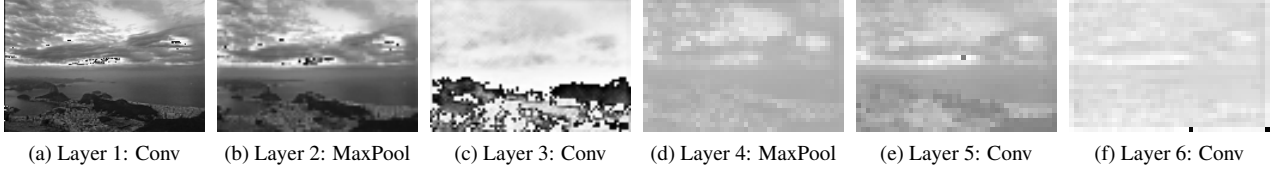


Figure 5: The List of Most Similar-to-Input IR Images at Each Hidden Layer — Extraction Model

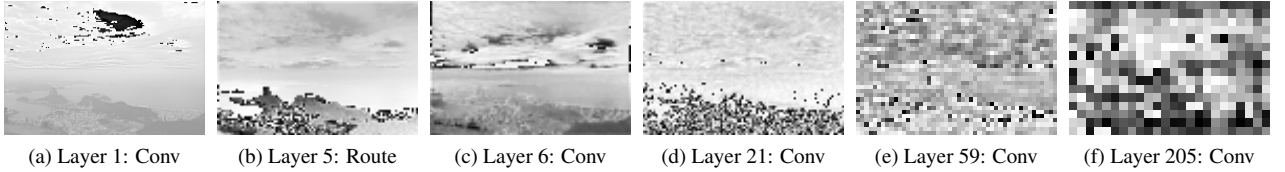


Figure 6: The List of Most Similar-to-Input IR Images at Each Hidden Layer — DenseNet Model

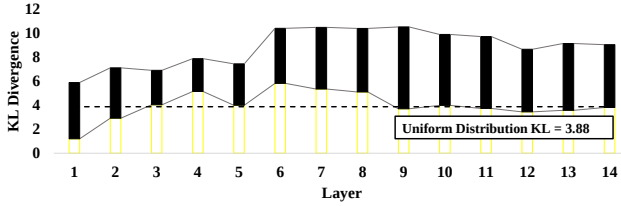


Figure 7: KL Divergence for Intermediate Representations of Hidden Layers — Darknet Reference Model

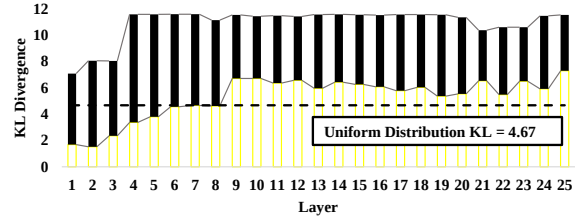


Figure 8: KL Divergence for Intermediate Representations of Hidden Layers — Extraction Model

representation $IR = \Phi_i(\mathbf{x})$, the adversary tends to compute an approximated inverse function ϕ^{-1} to generate an input $\tilde{\mathbf{x}} = \phi^{-1}(IR)$ that minimizes the distance between $\mathbf{x}$ and $\tilde{\mathbf{x}}$. Here we qualitatively review the state-of-the-art input reconstruction techniques for deep neural networks, analyze the requirements or preconditions for these research works, and demonstrate that *DeepEnclave* can protect the data privacy of user inputs from powerful adversaries equipped with these techniques.

Mahendran and Vedaldi [43] In this work, the authors proposed a gradient descent based approach to reconstructing original inputs by inverting the intermediate representations. Following the formal description of the input reconstruction problem above, the objective of

their approach is to minimize the loss function, which is the Euclid distance between $\Phi_i(\mathbf{x})$ and IR . Considering that Φ_i should not be uniquely invertible, they restrict the inversion by adding a regularizer to enforce natural image priors. In our design, the user-provisioned deep learning models are partially encrypted. The *FrontNet* models are encrypted by end users and are only allowed to be decrypted inside SGX enclaves. Assume $\mathcal{A}_2$ knows the input reconstruction technique from Mahendran and Vedaldi [43], the representation function Φ_i , which is equivalent to the *FrontNet* in our case, is not available in a decrypted form out of the enclave. In addition, we can also prevent the adversaries from querying the on-line *FrontNet* as a black-box to conduct their optimization procedure. The reason is that we use GCM to enable

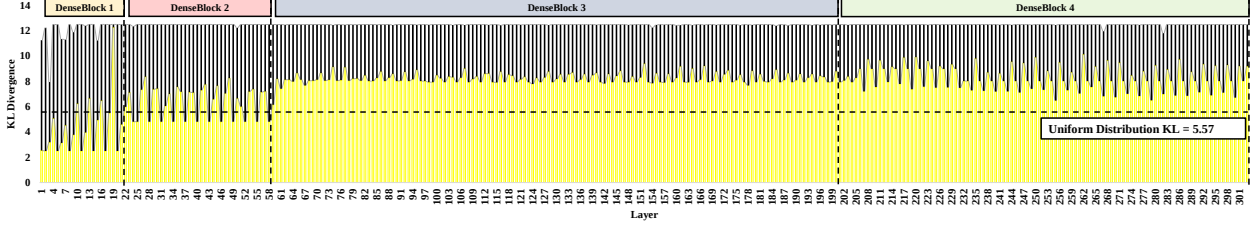


Figure 9: KL Divergence for Intermediate Representations of Hidden Layers — DenseNet Model

authenticated encryption. The enclave code can deny illegitimate requests, whose authentication tags cannot be verified correctly with end users’ symmetric keys. Thus $\mathcal{A}_2$ is not able to conduct the optimization procedure to compute both ϕ^{-1} and $\tilde{\mathbf{x}}$.

Dosovitskiy and Brox [18] This research has a similar goal of inverting the intermediate representations to reconstruct the original inputs. Compared to Mahendran and Vedaldi [43], the major difference is that they do not need to manually define the natural image priors, but learn the priors implicitly and generate reconstructed images with an up-convolutional neural network. They involve supervised training to build the up-convolutional neural network, in which the input is the intermediate representation $\Phi_i(\mathbf{x})$ and the target is the input $\mathbf{x}$. This reconstruction technique cannot work either in our security setting. $\mathcal{A}_2$ needs to collect the training pairs $\{\Phi_i(\mathbf{x}), \mathbf{x}\}$. Because the *FrontNet* is guaranteed to be confidential, i.e., the Φ_i is unknown, $\mathcal{A}_2$ cannot retrieve the model parameters and generate the training data to train the up-convolutional neural networks. Similarly, we can also prevent $\mathcal{A}_2$ from querying the online *FrontNet* as a black-box to generate training pairs, because $\mathcal{A}_2$ does not possess the end users’ keys to encrypt the inputs and generate correct authentication tags. Thus enclave code will not return IRs to $\mathcal{A}_2$. Without the up-convolutional neural network, $\mathcal{A}_2$ cannot reconstruct the original input either.

7 Implementation

We build our research prototype based on *Darknet* [50], which is an open source neural network implementation in C and CUDA. We support both user-provisioned encrypted inputs and encrypted *FrontNet*. End users can partition their DNNs at a specific layer first and encrypt both the *FrontNet* and inputs with tools provided by us on their local machines. We also implement a neural network assessment framework to measure the quality of IRs at different layers. It can guide end users to determine, for each specific deep learning model, the optimal number of layers to include in a *FrontNet* and run within

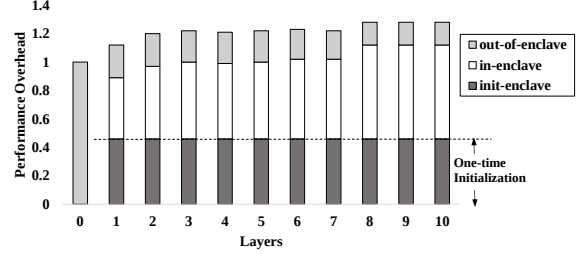


Figure 10: Performance Overhead of Running *FrontNet* in SGX Enclaves (compiled with `-O2`)

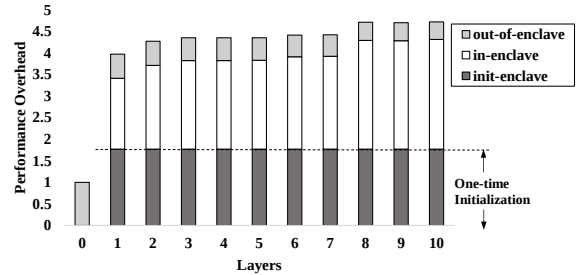


Figure 11: Performance Overhead of Running *FrontNet* in SGX Enclaves (compiled with `-Ofast`)

an enclave. In total, we add and revise 23,154 SLOC in C and 474 SLOC in Python for the development of *DeepEnclave*.

8 Performance Evaluation

In the performance evaluation, we measure the performance overhead for different system settings and indicate the constraint factors in practice. By understanding the trade-off between security and performance, end users can determine the level of security protection they tend to achieve and the corresponding performance and usability cost they may have to pay. Our testbed is equipped with an Intel i7-6700 3.40GHz CPU with 8 cores, 16GB of RAM, and running Ubuntu Linux 16.04 with kernel version 4.4.0.

We measure the inference performance of *DeepEnclave* by passing testing samples through the Extraction

Model. In the base case, we load the whole neural network without using SGX and obtain the average time for predicting these unencrypted images. To compare with the base case, we partition the network, load multiple layers as the *FrontNet* inside an SGX enclave and the following layers out of the enclave, and obtain the same performance metrics. We need to emphasize that both images and the *FrontNet* models are encrypted in these cases and are decrypted at runtime inside SGX enclaves. Due to the SGX memory limitation, we can load up to 10 layers of the Extraction Model into the enclave. We compiled *DeepEnclave* with both gcc optimization level -O2 and -Ofast (with -O3 and -ffast-math enabled) and present the normalized performance results in Figure 10 and 11 respectively. For each individual input, we include the one-time overhead of enclave initialization in the performance measurement. We distinguish the performance overhead contributed by enclave initialization, in-enclave computation, and out-of-enclave computation with bars of different colors. Layer 0 is the base case with unencrypted inputs and all layers run out of the SGX enclave.

For optimization level at -O2, we observe the performance overhead increase from 12% for running one layer inside an enclave to 28% for ten layers. Initializations of enclaves contribute to the most significant portion of the additional performance overhead. However, once the enclave is initialized, we observe that an inference task within an SGX enclave has even lower performance overhead as running out of the enclave. This is mainly due to the characteristic of deep neural network computation, which is computing-intensive and can benefit a lot from using the CPU cache and decrease the rate to read and write the encrypted memory, which is considered to be expensive in SGX. In the cloud scenario, we do not need to initialize and tear down an enclave for each service request, but can run one enclave as a long-time service to serve all client requests. For optimization level at -Ofast, we observe that the absolute time for enclave initialization is at the same level as in Figure 10. The in-enclave *FrontNet* computation causes 1.64x - 2.54x overhead compared to the base case. The *BackNet* still conduct inference computation at the same speed as the base case. We speculate that the slow down inside the enclave is due to the ineffective -ffast-math flag for floating arithmetic acceleration. We expect that in the future Intel will release optimized math library within SGX enclave to further reduce the floating arithmetic overhead.

Compared to cryptographic primitives based approaches [22, 42, 46] and running whole neural network within a single enclave [47], the performance overhead of *DeepEnclave* makes online privacy-preserving deep learning inference feasible and adaptable for production-level large-scale deep neural networks. The out-of-

enclave *BackNet* computation can still benefit from hardware acceleration and we grant the freedom to end users to adjust network partitioning strategy to satisfy their specific performance requirements.

9 Discussion

SGX Side-Channel Attacks Since the inception of SGX as a TEE with a strong threat model, i.e., to guarantee confidentiality and integrity of computation upon untrusted or adversarial system software, we have observed research studies on a spectrum of side-channels to extract sensitive information from enclave execution. These side-channels include page faults [66], high-resolution timers [27], branch history [39], memory management [65], CPU cache [13, 15, 24, 27], etc. In these case studies, side-channel attacks have been explored and applied to extract and further to infer sensitive information from various SGX-protected applications.

We have not observed any SGX side channel attacks to recover inputs of deep neural networks by now and we consider that such tasks are exceedingly difficult due to the special computation model of neural networks. For existing SGX side-channel attacks, the attackers need to eavesdrop the side-channel signals that leak the control transfer information. However, the memory access patterns of deep neural network computation do not depend on the input data. Thus it is impossible to recover input instance with the leaked side channel information. In addition, in our design for *DeepEnclave*, the *FrontNet* of the model is encrypted. This makes the prerequisites (availability of source code or binaries of target programs) for SGX side-channel attacks unsatisfied.

However, through side-channels, it is still possible to recover model hyperparameters, e.g., filter strides, activation function types, or the size of the inputs, which are pre-defined by deep learning model builders. Furthermore, if a deep learning image classification service needs to handle JPEG files as inputs, the file preprocessing procedure, i.e., the extraction of the pixel content of the images, may invoke some JPEG library functions, which have been demonstrated to be potentially vulnerable to side-channel attacks [27, 66].

Applicable Deep Learning Models *DeepEnclave* is based on the feature distillation process in DNNs. However, our system does not apply to these *information-preserving* neural networks designed for specific learning tasks. One representative case is the autoencoder [32], which is used for efficient encoding, dimension reduction, and learning generative models. No matter where we partition the autoencoder network, adversaries can always recover the (approximated) original in-

puts in any hidden layer. The reason is that autoencoder is trained to minimize the reconstruction errors between inputs and outputs. The original input information is always preserved (though could be compressed) in the autoencoder network. We leave the privacy protection for information-preserving neural networks as our future work.

10 Related Work

In this section we list the research efforts that are closely related to our work and highlight our unique contributions compared to these works.

Cryptographic Schemes Based Machine Learning

Most of the existing privacy-preserving machine learning solutions are based on cryptographic schemes, such as secure multi-party computation (SMC), fully homomorphic encryptions (FHE) [21], etc. Solutions based on SMC protect intermediate results of the computation when multiple parties perform collaborative machine learning on their private inputs. SMC has been used for several fundamental machine learning tasks [19, 35, 41, 46, 63, 64]. Besides these protocol-based solutions, recently researchers also propose to leverage cryptographic primitives to perform deep learning inference. Gilad-Bachrach et al. [22] proposed CryptoNets, a neural network model that makes predictions on data encrypted by FHE schemes. This approach protects the privacy of each individual input in terms of confidentiality. MiniONN [42] is an approach that transforms existing neural networks to an oblivious neural network that supports privacy-preserving predictions. Considering the significant performance overhead of using cryptographic schemes, we propose to leverage Intel SGX technology to keep part of the deep neural network computation in confidential on the cloud side. Hence we can protect the privacy of user inputs at inference and can defend against state-of-the-art input reconstruction attacks.

Distributed Deep Learning Shokri and Shmatikov [55] designed a distributed privacy-preserving deep learning protocol by sharing selective parameters and gradients for training deep neural network in a differentially private way. Ossia et al. [48] proposed a distributed machine learning architecture to protect the user’s input privacy. Their framework consists of a feature extractor on the mobile client side and a classifier on the server side. The server side performs inference task on the dimension-reduced extracted features from the mobile client. PrivyNet [40] is a splitting model deep learning training approach. They reused layers of pre-trained models for feature extraction on local machines and train

the cloud neural network for the learning tasks based on the feature representations generated by the local neural network. Different from their work, our approach leverages the TEE on the cloud directly to guarantee the confidentiality of user inputs and the user-provisioned model. Thus we significantly simplify the client’s logic and relieve client devices, which are supposed to have limited computing capacity and power usage restriction, from heavyweight neural network computation. In addition, our approach does not involve transferring intermediate representations through the network, thus eliminating the additional performance overhead for dimension reduction or data compression.

SGX Applications Researchers leverage SGX [16] technology to replace expensive cryptographic schemes for secure multi-party computation. Ohrimenko et al. [47] intended to leverage trusted SGX-enabled CPU for privacy-preserving multi-party machine learning. They also proposed to make machine learning algorithm data-oblivious, in order to prevent the SGX side-channel attacks. The SGX technique is also used for efficient two-party secure function evaluation [26], private membership test [61], trustworthy remote entity [38]. Different from the goals of these works that focus on the data sharing privacy in collaboration tasks, our work intends to protect the user input privacy from being exposed to cloud providers. SGX technology is also widely researched in cloud scenarios. VC3 [54] ran distributed MapReduce computation within SGX enclaves on the cloud to keep the confidentiality of user’s code and data. Opaque [69] was a distributed data analytics platform introducing SGX-enabled oblivious relational operators to mask data access patterns. SecureKeeper [14] provided an SGX-enhanced ZooKeeper to protect the sensitive application data. HardIDX [20] leveraged SGX to help search over encrypted data. Different in application scenarios, we intend to leverage SGX to protect the user input privacy and propose the SGX-enhanced partitioned deep learning model for the cloud services.

11 Conclusion

We systematically study the information exposure in deep learning inference and propose *DeepEnclave*, a privacy-enhancing deep learning system to minimize the sensitive information disclosure of user inputs. The key innovation of our system is to partition each deep neural network into two subnet models by exploiting the layered compositional network structure. We further leverage Intel SGX to protect the confidentiality of both user inputs and user-specified deep neural network layers. In addition, we design a neural network assessment framework

to quantify the privacy loss and can help end users determine the optimal partitioning layers for different neural network architectures. Our system by design can render existing state-of-the-art input reconstruction techniques ineffective, thus eliminating the channels for adversaries to invert the deep neural networks and reconstruct the user inputs.

References

- [1] Amazon AI. <https://aws.amazon.com/amazon-ai/>.
- [2] Darknet Reference Model. <https://pjreddie.com/darknet/imagenet/#reference>.
- [3] Extraction. <https://pjreddie.com/darknet/imagenet/#extraction>.
- [4] Google Cloud AI. <https://cloud.google.com/products/machine-learning/>.
- [5] Have I been pwned? <https://haveibeenpwned.com/>.
- [6] IBM Watson. <https://www.ibm.com/watson/>.
- [7] Intel Software Guard Extensions Remote Attestation End-to-End Example. <https://software.intel.com/en-us/articles/intel-software-guard-extensions-remote-attestation-end-to-end-example>.
- [8] Microsoft Azure Machine Learning. <https://azure.microsoft.com/en-us/services/machine-learning/>.
- [9] Wikipedia: List of data breaches. https://en.wikipedia.org/wiki/List_of_data_breaches.
- [10] ANATI, I., GUERON, S., JOHNSON, S., AND SCARLATA, V. Innovative technology for cpu based attestation and sealing. In *Proceedings of the 2nd international workshop on hardware and architectural support for security and privacy* (2013).
- [11] BENGIO, Y., SIMARD, P., AND FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks* 5, 2 (1994), 157–166.
- [12] BOIVIE, R., AND WILLIAMS, P. Secureblue++: Cpu support for secure executables. Tech. rep., Research report, IBM, 2013.
- [13] BRASSER, F., MÜLLER, U., DMITRIENKO, A., KOSTIAINEN, K., CAPKUN, S., AND SADEGHI, A. Software grand exposure: SGX cache attacks are practical. *CoRR abs/1702.07521* (2017).
- [14] BRENNER, S., WULF, C., GOLTZSCHE, D., WEICHBRODT, N., LORENZ, M., FETZER, C., PIETZUCH, P. R., AND KAPITZA, R. Securekeeper: Confidential zookeeper using intel SGX. In *Proceedings of the 17th International Middleware Conference* (2016).
- [15] CHEN, G., CHEN, S., XIAO, Y., ZHANG, Y., LIN, Z., AND LAI, T. H. Sgxpectre attacks: Leaking enclave secrets via speculative execution. *arXiv preprint arXiv:1802.09085* (2018).
- [16] COSTAN, V., AND DEVADAS, S. Intel sgx explained. *IACR Cryptology ePrint Archive* (2016), 86.
- [17] DENG, J., DONG, W., SOCHER, R., LI, L.-J., LI, K., AND FEI-FEI, L. Imagenet: A large-scale hierarchical image database. In *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on* (2009), IEEE, pp. 248–255.
- [18] DOSOVITSKIY, A., AND BROX, T. Inverting visual representations with convolutional networks. In *IEEE Conference on Computer Vision and Pattern Recognition* (2016).
- [19] DU, W., HAN, Y. S., AND CHEN, S. Privacy-preserving multivariate statistical analysis: Linear regression and classification. In *Proceedings of the 2004 SIAM international conference on data mining* (2004).
- [20] FUHRY, B., BAHMANI, R., BRASSER, F., HAHN, F., KERSCHBAUM, F., AND SADEGHI, A. Hardidx: Practical and secure index with SGX. In *Data and Applications Security and Privacy XXXI: 31st Annual IFIP WG 11.3 Conference* (2017).
- [21] GENTRY, C. *A fully homomorphic encryption scheme*. Stanford University, 2009.
- [22] GILAD-BACHRACH, R., DOWLIN, N., LAINE, K., LAUTER, K. E., NAEHRIG, M., AND WERNING, J. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *Proceedings of the 33rd International Conference on Machine Learning*, (2016).
- [23] GLOROT, X., AND BENGIO, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (2010), pp. 249–256.
- [24] GÖTZFRIED, J., ECKERT, M., SCHINZEL, S., AND MÜLLER, T. Cache attacks on intel SGX. In *Proceedings of the 10th European Workshop on Systems Security* (2017).
- [25] GRAVES, A., MOHAMED, A., AND HINTON, G. E. Speech recognition with deep recurrent neural networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing* (2013).
- [26] GUPTA, D., MOOD, B., FEIGENBAUM, J., BUTLER, K., AND TRAYNOR, P. Using intel software guard extensions for efficient two-party secure function evaluation. In *International Conference on Financial Cryptography and Data Security* (2016).
- [27] HÄHNEL, M., CUI, W., AND PEINADO, M. High-resolution side channels for untrusted operating systems. In *2017 USENIX Annual Technical Conference* (2017).
- [28] HANNUN, A. Y., CASE, C., CASPER, J., CATANZARO, B., DI-AMOS, G., ELSER, E., PRENGER, R., SATHEESH, S., SENGUPTA, S., COATES, A., AND NG, A. Y. Deep speech: Scaling up end-to-end speech recognition. *CoRR abs/1412.5567* (2014).
- [29] HE, K., WANG, Y., AND HOPCROFT, J. A powerful generative model using random weights for the deep image representation. In *Advances in Neural Information Processing Systems* (2016).
- [30] HE, K., ZHANG, X., REN, S., AND SUN, J. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition* (2016).
- [31] HINTON, G., DENG, L., YU, D., DAHL, G. E., R. MOHAMED, A., JAITLY, N., SENIOR, A., VANHOUCHE, V., NGUYEN, P., SAINATH, T. N., AND KINGSBURY, B. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine* (2012).
- [32] HINTON, G. E., AND SALAKHUTDINOV, R. R. Reducing the dimensionality of data with neural networks. *science* 313, 5786 (2006), 504–507.
- [33] HUANG, G., LIU, Z., VAN DER MAATEN, L., AND WEINBERGER, K. Q. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2017).
- [34] HUNT, T., SONG, C., SHOKRI, R., SHMATIKOV, V., AND WITCHEL, E. Chiron: Privacy-preserving machine learning as a service. *arXiv preprint arXiv:1803.05961* (2018).
- [35] JAGANNATHAN, G., AND WRIGHT, R. N. Privacy-preserving distributed k-means clustering over arbitrarily partitioned data. In *Proceedings of the eleventh ACM SIGKDD international conference on Knowledge discovery in data mining* (2005).

- [36] KAPLAN, D., POWELL, J., AND WOLLER, T. Amd memory encryption. Tech. rep., 2016.
- [37] KRIZHEVSKY, A., SUTSKEVER, I., AND HINTON, G. E. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems* (2012).
- [38] KÜÇÜK, K. A., PAVERD, A., MARTIN, A., ASOKAN, N., SIMPSON, A., AND ANKELE, R. Exploring the use of intel sgx for secure many-party applications. In *Proceedings of the 1st Workshop on System Software for Trusted Execution* (2016).
- [39] LEE, S., SHIH, M.-W., GERA, P., KIM, T., KIM, H., AND PEINADO, M. Inferring fine-grained control flow inside sgx enclaves with branch shadowing. In *26th USENIX Security Symposium* (2017).
- [40] LI, M., LAI, L., SUDA, N., CHANDRA, V., AND PAN, D. Z. Privynet: A flexible framework for privacy-preserving deep neural network training with a fine-grained privacy control. *arXiv preprint arXiv:1709.06161* (2017).
- [41] LINDELL, Y., AND PINKAS, B. Privacy preserving data mining. In *Advances in Cryptology CRYPTO 2000* (2000).
- [42] LIU, J., JUUTI, M., LU, Y., AND ASOKAN, N. Oblivious neural network predictions via minionn transformations. In *ACM Conference on Computer and Communications Security (CCS)* (2017).
- [43] MAHENDRAN, A., AND VEDALDI, A. Understanding deep image representations by inverting them. In *IEEE Conference on Computer Vision and Pattern Recognition* (2015).
- [44] MCKEEN, F., ALEXANDROVICH, I., BERENZON, A., ROZAS, C. V., SHAFI, H., SHANBHOGUE, V., AND SAVAGAONKAR, U. R. Innovative instructions and software model for isolated execution. In *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy* (2013).
- [45] MCMAHAN, H. B., MOORE, E., RAMAGE, D., HAMPSON, S., ET AL. Communication-efficient learning of deep networks from decentralized data. *arXiv preprint arXiv:1602.05629* (2016).
- [46] MOHASSEL, P., AND ZHANG, Y. Secureml: A system for scalable privacy-preserving machine learning. In *38th IEEE Symposium on Security and Privacy* (2017).
- [47] OHRIMENKO, O., SCHUSTER, F., FOURNET, C., MEHTA, A., NOWOZIN, S., VASWANI, K., AND COSTA, M. Oblivious multi-party machine learning on trusted processors. In *USENIX Security Symposium* (2016).
- [48] OSSIA, S. A., SHAMSABADI, A. S., TAHERI, A., RABIEE, H. R., LANE, N., AND HADDADI, H. A hybrid deep learning architecture for privacy-preserving mobile analytics. *arXiv preprint arXiv:1703.02952* (2017).
- [49] PAN, S. J., AND YANG, Q. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering* (2010).
- [50] REDMON, J. Darknet: Open source neural networks in c. <http://pjreddie.com/darknet/>, 2013–2016.
- [51] SALTAFORMAGGIO, B., BHATIA, R., GU, Z., ZHANG, X., AND XU, D. Guitar: Piecing together android app guis from memory images. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security* (2015).
- [52] SALTAFORMAGGIO, B., BHATIA, R., GU, Z., ZHANG, X., AND XU, D. Vcr: App-agnostic recovery of photographic evidence from android device memory images. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security* (2015).
- [53] SALTAFORMAGGIO, B., GU, Z., ZHANG, X., AND XU, D. Dscrite: Automatic rendering of forensic information from memory images via application logic reuse. In *USENIX Security Symposium* (2014).
- [54] SCHUSTER, F., COSTA, M., FOURNET, C., GKANTSIDIS, C., PEINADO, M., MAINAR-RUIZ, G., AND RUSSINOVICH, M. Vc3: Trustworthy data analytics in the cloud using sgx. In *Security and Privacy (SP), 2015 IEEE Symposium on* (2015).
- [55] SHOKRI, R., AND SHMATIKOV, V. Privacy-preserving deep learning. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security* (2015), ACM.
- [56] SHWARTZ-ZIV, R., AND TISHBY, N. Opening the black box of deep neural networks via information. *arXiv preprint arXiv:1703.00810* (2017).
- [57] SILVER, D., HUANG, A., MADDISON, C. J., GUEZ, A., SIFRE, L., VAN DEN DRIESSCHE, G., SCHRITTWIESER, J., ANTONOGLU, I., PANNEERSHELVA, V., LANCTOT, M., DIELEMAN, S., GREWE, D., NHAM, J., KALCHBRENNER, N., SUTSKEVER, I., LILICRAP, T. P., LEACH, M., KAVUKCUOGLU, K., GRAEPEL, T., AND HASSABIS, D. Mastering the game of go with deep neural networks and tree search. *Nature* (2016).
- [58] SILVER, D., SCHRITTWIESER, J., SIMONYAN, K., ANTONOGLU, I., HUANG, A., GUEZ, A., HUBERT, T., BAKER, L., LAI, M., BOLTON, A., ET AL. Mastering the game of go without human knowledge. *Nature* 550, 7676 (2017), 354–359.
- [59] SIMONYAN, K., AND ZISSERMAN, A. Very deep convolutional networks for large-scale image recognition. *CoRR abs/1409.1556* (2014).
- [60] SUTSKEVER, I., VINYALS, O., AND LE, Q. V. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems* (2014), pp. 3104–3112.
- [61] TAMRAKAR, S., LIU, J., PAVERD, A., EKBERG, J.-E., PINKAS, B., AND ASOKAN, N. The circle game: Scalable private membership test using trusted hardware. In *Proceedings of the 2017 ACM Conference on Computer and Communications Security* (2017).
- [62] TRAMER, F., AND BONEH, D. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287* (2018).
- [63] VAIDYA, J., AND CLIFTON, C. Privacy preserving association rule mining in vertically partitioned data. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining* (2002).
- [64] VAIDYA, J., KANTARCIOĞLU, M., AND CLIFTON, C. Privacy-preserving naive bayes classification. *The VLDB Journal/The International Journal on Very Large Data Bases* (2008).
- [65] WANG, W., CHEN, G., PAN, X., ZHANG, Y., WANG, X., BINDSCHAEDLER, V., TANG, H., AND GUNTER, C. A. Leaky cauldron on the dark land: Understanding memory side-channel hazards in sgx. In *Proceedings of the 24th ACM SIGSAC conference on computer and communications security* (2017).
- [66] XU, Y., CUI, W., AND PEINADO, M. Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In *Security and Privacy (SP), 2015 IEEE Symposium on* (2015).
- [67] YOSINSKI, J., CLUNE, J., NGUYEN, A. M., FUCHS, T. J., AND LIPSON, H. Understanding neural networks through deep visualization. *CoRR abs/1506.06579* (2015).
- [68] ZEILER, M. D., AND FERGUS, R. Visualizing and understanding convolutional networks. In *Computer Vision - ECCV 2014 - 13th European Conference* (2014).
- [69] ZHENG, W., DAVE, A., BEEKMAN, J. G., POPA, R. A., GONZALEZ, J. E., AND STOICA, I. Opaque: An oblivious and encrypted distributed analytics platform. In *14th USENIX Symposium on Networked Systems Design and Implementation* (2017).